

SIEVE：多步智能体中的观察准入与可靠状态修正

上次会议遗留问题：

- 问题场景改为“agent的多步决策”，聚焦当下比较火热的agent领域
- 方法论：不可修改模型的注意力层，容易导致模型“失语”，输出乱码

方法全称： Selective Information Evaluation and Verification for Executable Beliefs

研究定位： 不是“如何让 Agent 检索更多信息”，而是“已经观察到的信息是否有资格进入 Agent 后续行动所依赖的可信任任务状态”。

0. 汇报内容

0.1 论文要解决的核心问题【重点】

一个多步 Agent 在执行任务时，会不断接收用户消息、工具返回、网页内容、长期记忆和环境反馈【什么形式存在】。现有 Agent 通常把这些新观察直接追加到上下文，或者直接交给语言模型生成新的状态摘要。

这种做法隐含了一个危险假设：

只要一条信息被 Agent 看见，它就可以被后续行动使用。

但新观察可能是过时的、错误的、条件不适用的、来源不可靠的，或者虽然真实却与当前任务无关。如果这些观察未经判断便改变 Agent 的任务状态，错误会继续影响后续规划、工具调用和不可逆动作。

本文将这一现象定义为：

观察诱导的状态污染（Observation-Induced State Contamination）： 不应被采纳的新观察错误进入 Agent 的可信任任务状态，并进一步改变后续行动。

因此，本文的核心问题是：

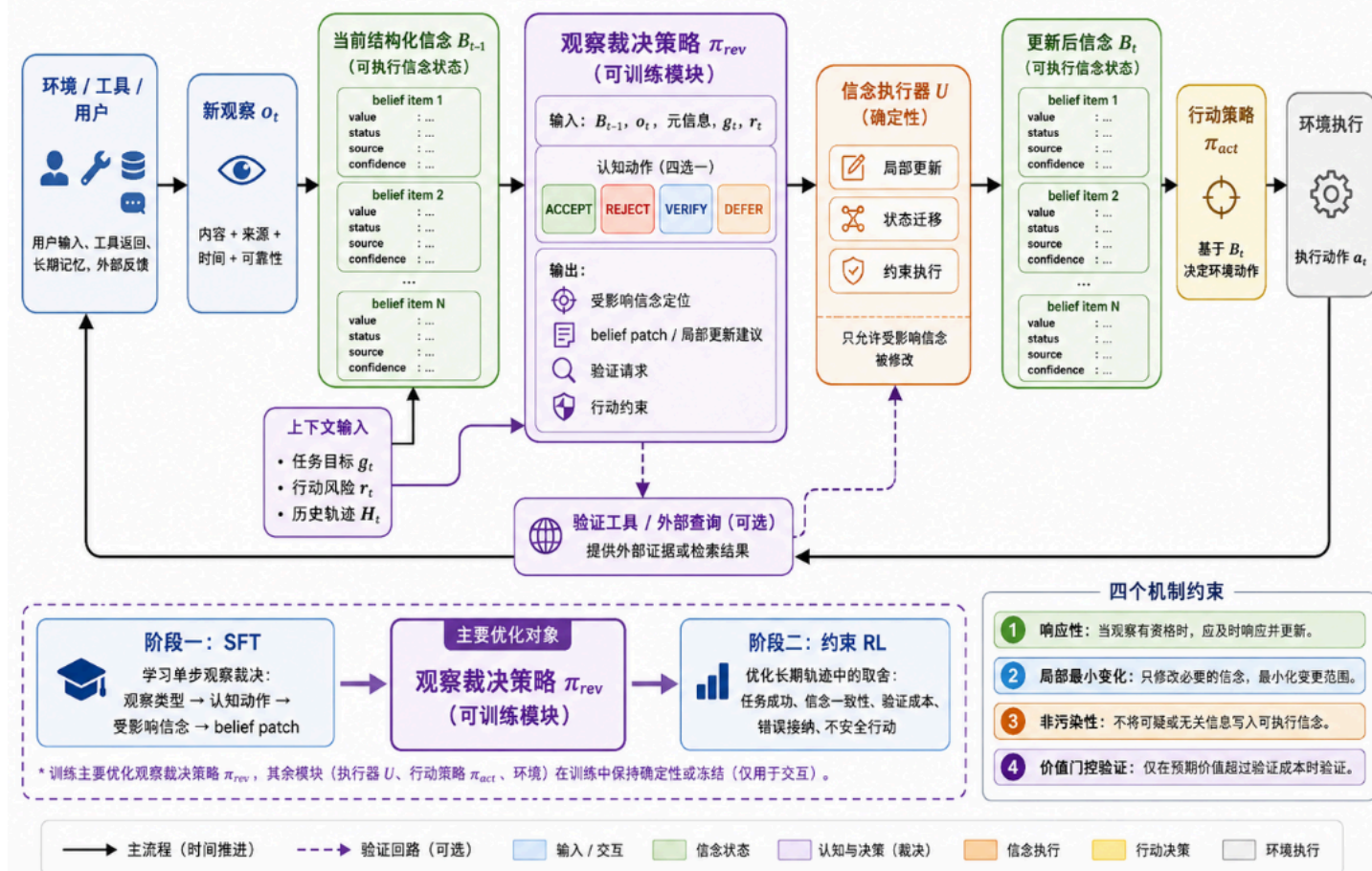
Agent 收到一条新观察时，应该立即更新当前任务状态，暂时保留并验证，还是忽略它？

0.2 SIEVE 方法【重点】

📌 【信念】不是模型参数知识或主观观点，而是 Agent 根据当前观察、历史记忆和工具反馈，对任务相关隐藏环境状态形成的暂时估计。进一步将其中可以直接支持后续行动的部分称为可执行信念状态。📖 信念概念

【形式：外部是可读、可修改的结构化信念；内部计算时，模型会把它编码成高维向量】

核心思想：在新观察与环境行动之间加入可训练的观察裁决策略，先判断观察是否有资格改变当前可执行信念，再执行局部更新或触发验证，并据此决定后续行动。



SIEVE 在新观察与任务状态更新之间加入一个可学习的 **State Revision Policy**（状态修正策略）。

该策略只做三个决定：

- UPDATE**：证据可靠、相关且适用，用它局部更新任务状态；
- HOLD**：证据可能有用，但当前不足以承诺，保留候选值并视情况发起验证；
- IGNORE**：证据过时、无关、低可信或条件不适用，不修改任务状态。

原始设计中的 **ACCEPT / REJECT / VERIFY / DEFER** 可以被收敛为这三个更基础的状态修正动作：

原认知动作	SIEVE 动作	解释
ACCEPT	UPDATE	允许观察进入可信状态
REJECT	IGNORE	不允许观察改变可信状态
VERIFY	HOLD + 查询	暂不承诺，同时主动获取证据
DEFER	HOLD + 暂缓	暂不承诺，当前也没有值得执行的验证

这使论文不需要同时训练四个含义部分重叠的宏动作。真正需要学习的本体是：

新观察是否应当改变当前可信任任务状态。

0.3 训练的到底是什么【重点】

两阶段训练始终训练同一个对象：

State Revision Policy，也就是 observation-to-state transition policy。

它既可以从系统角度称为一个“模块”，也可以从机器学习角度称为一个“策略”。

- 从系统角度看，它是插在 observation 和 state update 之间的 Revision Adapter；
- 从学习角度看，它是一个条件策略函数，学习在不同状态下选择 UPDATE / HOLD / IGNORE，并预测局部状态补丁。

形式化为：

$$\pi_{\text{rev}} : (B_{t-1}, o_t, m_t, g_t, r_t) \rightarrow (z_t, A_t, \Delta B_t, q_t),$$

其中：

- B_{t-1} ：更新前的可信任任务状态；
- o_t ：新观察；
- m_t ：来源、时间、适用条件等元信息；
- g_t ：当前任务目标；
- r_t ：依赖该状态的候选动作风险；
- $z_t \in \text{UPDATE, HOLD, IGNORE}$ ；
- A_t ：受影响字段集合；
- ΔB_t ：局部 JSON patch；
- q_t ：HOLD 时可选的验证请求。

不需要重新训练完整 Agent。推荐实现是：

冻结 Action Agent，只训练一个 Revision LoRA/Adapter，并使用确定性程序执行状态补丁。

SFT 学习的是一个**初始策略**:

$$\pi_{\text{base}} \rightarrow \pi_{\text{SFT}}.$$

它通过模仿标注数据中的正确裁决行为，学会基本的单步决策规则。

使用监督微调学习观察裁决策略的初始行为，LoRA 仅作为参数高效训练方式。

1. 从原生 Agent 到问题定位

1.1 原生 Agent 的基本数据流

一个常见工具型 Agent 可以简化为：

1. 接收用户目标；
2. 根据上下文规划下一步；
3. 调用工具或与环境交互；
4. 接收新观察；
5. 将新观察追加到上下文或状态摘要；
6. 根据更新后的上下文生成下一动作。

普通 Agent 的动作策略可以写成：

$$a_t \sim \pi_{\text{act}}(a_t \mid H_{t-1}, o_t, g_t),$$

其中 H_{t-1} 是此前完整轨迹， o_t 是刚收到的新观察。

这意味着原始观察可以直接改变动作分布：

$$o_t \longrightarrow a_t.$$

当 o_t 包含错误、过时、冲突或条件不适用的信息时，Agent 缺少一个明确的“是否采纳”边界。

1.2 问题不只是检索错误

检索模块回答的是：

哪些信息应该被展示给 Agent？

SIEVE 回答的是：

已经展示给 Agent 的信息，哪些可以改变后续行动所依赖的状态？

二者并不等价。

一条信息可以被正确检索，但仍然不应该被更新到任务状态。例如：

- 检索到的是旧版本订单状态；
- 长期记忆来自另一位用户；
- 工具返回值与官方状态冲突；
- 一条事实本身正确，但只适用于另一个时间或条件；
- 观察与当前任务相关，却不足以支持高风险操作。

因此，正确的数据流应当是：

Observation $\rightarrow$ Admission Decision $\rightarrow$ State Revision $\rightarrow$ Action.

而不是：

Observation $\rightarrow$ Action.

1.3 SIEVE 改变了什么

SIEVE 将 Agent 数据流改为：

π_{rev}

$(B_{t-1}, o_t, m_t, g_t, r_t),$

$U(B_{t-1}, z_t, A_t, \Delta B_t),$

$a_t \sim \pi_{\mathrm{act}}(a_t \mid B_t, g_t).$

其中 U 是确定性状态执行器，只允许执行合法的局部 patch。

核心变化不是简单增加一个分类标签，而是切断：

$o_t \longrightarrow a_t$

这条未经审查的直接路径，使新观察只能通过受约束的可信状态影响行动。

2. 结构化可信任务状态

2.1 为什么不能继续使用自由文本摘要

如果每一步都让 LLM 用自然语言重写完整状态，例如：

用户似乎已经付款，可能可以发货；之前的订单状态也许已经过时……

这种状态存在三个问题：

- 1. 很难判断具体哪个字段发生了变化；
- 2. 模型可能在更新一个字段时顺手修改无关内容；
- 3. 很难直接验证更新是否符合最小变化原则。

因此，SIEVE 使用结构化任务状态，而不是自由文本 belief summary。

2.2 最小状态 Schema

每个状态项表示为：

\langle

id_i,

value_i,

status_i,

source_i,

time_i

\rangle.

第一版只保留五个必要字段：

字段	含义	示例
id	状态字段名称	payment_status
value	当前可信值	unpaid
status	当前承诺状态	trusted / pending_verification
source	当前值的来源	official_payment_api
time	观察时间或内容有效时间	2026-07-15T14:20

示例：

代码块

```
1  {
2    "id": "payment_status",
3    "value": "unpaid",
4    "status": "trusted",
5    "source": "official_payment_api",
```

```
6     "time": "2026-07-15T14:20:00+08:00"
7 }
```

第一版不需要同时引入复杂置信度传播、完整知识图谱和多级 belief lifecycle。这样可以确保论文贡献集中在 **状态更新准入策略**，而不是状态表示本身。

2.3 新观察的元信息

新观察表示为：

```
\langle
content_t,
source_t,
observed_time_t,
valid_time_t,
condition_t
```

```
\rangle.
```

例如：

代码块

```
1  {
2    "content": "订单已经支付",
3    "source": "cached_web_page",
4    "observed_time": "2026-07-15T14:31:00+08:00",
5    "valid_time": "unknown",
6    "condition": "current_order"
7  }
```

SIEVE 不要求上游系统一定给出完美元信息。元信息缺失本身就是一个输入特征，可以推动策略选择 **HOLD**，而不是默认 **UPDATE**。

3. SIEVE 方法

3.1 Revision Policy 的输出

Revision Adapter 一次输出以下结构：

代码块

```
1  {
2    "decision": "HOLD",
3    "affected_fields": ["payment_status"],
4    "patch": [
5      {
6        "op": "SET_STATUS",
7        "id": "payment_status",
8        "value": "pending_verification"
9      }
10   ],
11   "verification_request": {
12     "tool": "official_payment_api",
13     "field": "payment_status"
14   },
15   "reason_type": "source_conflict"
16 }
```

模型不是只输出一个三分类标签，还需要联合预测：

- 观察处置决定；
- 受影响字段；
- 局部 patch；
- 可选验证请求。

因此，SIEVE 是一个结构化状态修正策略，而不是普通文本过滤器。

3.2 三种状态修正动作

UPDATE

适用于：

- 观察与当前目标相关；
- 来源达到可信要求；
- 时间上仍有效；
- 条件适用于当前对象；
- 新观察与当前状态一致，或足以替代旧值。

执行效果：

- 更新指定字段；
- 保存来源和时间；

- 不允许修改未列入 `affected_fields` 的字段。

HOLD

适用于：

- 新观察可能影响任务；
- 当前证据不足以接受或拒绝；
- 与可信状态发生冲突；
- 时间或适用条件不明确；
- 依赖该状态的后续动作风险较高。

执行效果：

- 保留旧可信值；
- 将对应字段标记为 `pending_verification`；
- 有正价值验证渠道时发起查询；
- 没有合适验证渠道时暂缓依赖该字段的高风险动作。

IGNORE

适用于：

- 观察与任务无关；
- 信息已经过时；
- 条件不适用于当前对象；
- 来源明显不可靠；
- 已被更高可信证据否定。

执行效果：

- 不修改可信状态；
- 可以把观察记录到审计日志；
- 不允许它直接影响行动。

3.3 确定性 State Executor

为了避免语言模型自由重写完整状态，SIEVE 将语义判断与状态执行分开。

模型负责：

- 选择动作；
- 定位字段；

- 生成建议 patch。

确定性执行器负责：

- 校验 JSON；
- 检查操作是否合法；
- 限制 patch 的作用范围；
- 执行状态更新；
- 拦截依赖待确认字段的高风险动作。

允许的最小操作集合为：

代码块

```
1  ADD_FIELD
2  SET_VALUE
3  SET_STATUS
```

硬约束为：

1. `IGNORE` 时 patch 必须为空；
2. `HOLD` 只能设置 `pending_verification`，不能覆盖可信值；
3. `UPDATE` 只能修改 `affected_fields` 中的字段；
4. 所有未受影响字段保持不变；
5. 高风险动作若依赖 `pending_verification` 字段，则暂不执行。

这使方法形成：

学习型语义裁决 + 确定性结构约束。

4. 为什么 Prompt 能做，但仍然需要训练

4.1 Prompt-only 版本完全可以实现

可以给原生 Agent 一个 Prompt：

代码块

```
1  在执行动作前，请先判断新观察应该：
2  1. 更新任务状态；
3  2. 暂时保留并验证；
4  3. 忽略。
```

因此，Prompt-only SIEVE 必须作为强基线。论文不能通过“原生 Agent 没有被提醒做裁决”来制造性能差距。

4.2 Prompt 是流程提示，不是稳定策略

Prompt 可以让模型在某些样本上模仿上述流程，但它通常无法保证：

- 不同表达方式下决策一致；
- 长轨迹中一直遵循相同准入边界；
- 不因上下文长度增加而忽略元信息；
- 不自由改写无关字段；
- 不出现全更新、全忽略或全验证等退化行为。

训练的价值不在于让模型第一次知道三个动作，而是学习稳定的条件决策边界：

$$(B, o, m, g, r) \longrightarrow P(z \mid B, o, m, g, r).$$

4.3 Prompt 很难学习延迟后果

例如：

- 第 2 步错误 `UPDATE` 了一条工具观察；
- 第 4 步基于污染状态生成错误计划；
- 第 8 步执行不可逆操作后任务失败。

最终失败的根因发生在第 2 步，而不是第 8 步。

单步 Prompt 很难利用完整轨迹结果调整早期准入决策。轨迹训练的价值在于：

将未来任务失败、额外验证成本和不安全动作，归因到早期的状态修正选择。

4.4 训练必须通过可证伪实验获得合理性

论文必须比较：

1. Frozen Vanilla Agent；
2. Prompt-only SIEVE；
3. SFT-SIEVE；
4. SFT + Trajectory Optimization SIEVE；

5. Oracle Revision Policy。

如果强 Prompt 已经接近 Oracle，或者 SFT/RL 没有在相同预算下显著改善性能，则训练不应被包装为核心贡献。

因此，训练不是预设结论，而是一个可证伪假设：

显式 Prompt 可以引导观察裁决，但无法稳定学习状态污染风险、验证成本和长期行动收益之间的决策边界。

5. 两阶段训练方案

5.1 训练对象保持不变

两个阶段都只优化 Revision Adapter：

π_{rev} 。

原则上冻结：

- Action Agent；
- 工具集合；
- 环境；
- State Executor。

这样性能变化才能归因于 observation-to-state transition，而不是完整 Agent 被重新训练后整体能力增强。

5.2 第一阶段：多任务 SFT

训练目标

SFT 学习单步结构化裁决：

$$(B_{t-1}, o_t, m_t, g_t, r_t) \rightarrow (z_t, A_t, \Delta B_t, q_t).$$

它解决的是：

模型是否知道，在给定状态、观察、来源、时间和风险时，应该更新、暂存还是忽略？

损失函数

$$\begin{aligned} & \mathcal{L}_{\text{decision}} \\ & + \\ & \alpha_A \mathcal{L}_{\text{affected}} \\ & + \end{aligned}$$

$$\alpha_P \mathcal{L}_{\mathrm{patch}} + \alpha_Q \mathcal{L}_{\mathrm{query}}.$$

其中：

- $\mathcal{L}_{\text{decision}}$: `UPDATE / HOLD / IGNORE` 分类损失；
- $\mathcal{L}_{\text{affected}}$: 受影响字段定位损失；
- $\mathcal{L}_{\text{patch}}$: 合法 JSON patch 生成损失；
- $\mathcal{L}_{\text{query}}$: 验证工具及查询字段选择损失。

数据设计

推荐以有真实环境状态的干净轨迹为基础，构造配对反事实样本。保持任务目标和语言难度基本不变，只修改一个因素：

- 事实真假；
- 时间新旧；
- 来源可信度；
- 条件适用性；
- 是否与当前状态冲突；
- 验证工具是否可用；
- 下游动作风险。

这种 paired counterfactual 设计可以减少模型利用表面词汇判断标签。

5.3 第二阶段：轨迹级策略优化

为什么 SFT 之后仍可能有问题

一个单步准确率很高的 SFT 模型，仍可能在长轨迹中出现：

- 过度 `HOLD`，导致频繁验证；
- 过度 `IGNORE`，错过重要更新；
- 过度 `UPDATE`，造成状态污染；
- 在低风险任务中过于保守；
- 在高风险不可逆动作前不够保守。

因此，第二阶段不再学习三个动作的基本含义，而是学习它们在长期任务中的取舍。

推荐顺序

为了避免为了论文强行使用在线 RL，推荐按以下顺序推进：

1. 先做 SFT；
 2. 检查是否存在过度验证、长期信用分配和成本权衡问题；
 3. 优先尝试离线偏好优化；
 4. 只有延迟后果明显时，再使用在线约束强化学习。
-

5.4 离线偏好优化

对同一个状态采样多种裁决轨迹，构造偏好对：

- 任务成功相同时，验证更少的轨迹更优；
- 验证成本相同时，false update 更少的轨迹更优；
- 都能完成任务时，不安全动作更少的轨迹更优；
- 同样安全时，过度等待更少的轨迹更优。

可采用 DPO、IPO 或排序损失优化 Revision Adapter。

其优势是训练稳定、成本较低，并且不需要完整在线环境反复采样。

5.5 约束强化学习

当早期错误的后果会在多步以后出现时，使用轨迹级约束强化学习。

主目标为：

$$\max_{\pi_{\text{rev}}} \mathbb{E} [R_{\text{task}} + \beta R_{\text{state}}] .$$

同时满足以下约束：

$$\mathbb{E}[C_{\text{false update}}] \leq \epsilon_{\text{fu}},$$

$$\mathbb{E}[C_{\text{unsafe}}] \leq \epsilon_{\text{safe}},$$

$$\mathbb{E}[C_{\text{verify}}] \leq B_{\text{verify}},$$

$$\mathbb{E}[C_{\text{hold}}] \leq B_{\text{hold}}.$$

含义分别为：

- 无效观察被更新的比例不能超过阈值；
- 不安全或不可逆错误行动不能超过阈值；
- 验证调用不能超过预算；
- **HOLD** 不能成为无限拖延策略。

对应的拉格朗日目标为：

λ_{h}

$(C_{\mathrm{hold}} - B_{\mathrm{hold}})$

$\big]$.

$\end{aligned}$

无效公式

- ϵ_{fu} 表示可接受的污染风险；
- B_{verify} 表示系统允许的验证预算；
- B_{hold} 表示可接受的延迟程度；
- 乘子 λ 由训练过程自适应调整，而不是完全依赖人工调参。

6. 理论机制与可检验性质

SIEVE 不声称 LLM 能够在开放世界中始终判断自然语言真假，也不声称完整满足 AGM 或实现 Bayes-optimal belief revision。

理论部分只证明：

在 Revision Policy 给出某个裁决结果后，受约束 State Executor 能保证哪些结构性质；模型误差如何转化为可测量的状态污染风险。

6.1 性质一：响应性

当一条有效、相关且足够可信的观察被正确判定为 UPDATE 时，受影响字段应当发生对应变化。

若真实受影响集合为 A_t^* ，且模型正确预测：

$$z_t = \text{UPDATE}, \quad A_t = A_t^*,$$

则更新执行器应满足：

$$\exists c_i \in A_t, \quad c_i^t \neq c_i^{t-1}.$$

该性质保证模型不会口头表示“接受新证据”，却不实际修改任务状态。

可测指标：

- valid update recall;
- stale-state correction rate;
- update effectiveness;
- observation 后的动作适应率。

6.2 性质二：局部最小变化

若执行器只允许修改 A_t 中的字段，则：

$$\forall c_j \notin A_t, \quad c_j^t = c_j^{t-1}.$$

这是一条由程序直接保证的硬不变量。

但模型可能预测错误的 A_t 。因此，论文需要同时报告：

- affected-set precision / recall;
- collateral edit rate;
- unaffected field preservation。

结构保证与学习误差必须明确分开。

6.3 性质三：非污染性

当决策为 `IGNORE` 时：

$$z_t = \text{IGNORE} \Rightarrow B_t = B_{t-1}.$$

当决策为 `HOLD` 时，新候选值不能覆盖旧可信值：

$$z_t = \text{HOLD} \Rightarrow \text{value}_t(c) = \text{value}_{t-1}(c),$$

$$\text{status}_t(c) = \text{pending}_{\text{verification}}.$$

因此，一条尚未被采纳的观察不能直接进入可信状态。

若无效观察被错误更新的概率为：

$$\begin{aligned} &P \\ &\left(\right. \\ &z_t = \mathrm{UPDATE} \\ &\mid \\ &o_t \setminus \text{不应更新} \\ &\left. \right), \end{aligned}$$

则其进入可信状态的概率满足：

$$\begin{aligned} &P \\ &\left(\right. \\ &o_t \setminus \text{污染} \mid B_t \\ &\mid \end{aligned}$$

o_t\ \text{不应更新}

\right)

\le

\epsilon_{\mathrm{fu}}.

这使系统污染风险可以由 $false - updatarate$ 上界控制。

6.4 性质四：价值门控验证

不是所有不确定观察都值得调用工具验证。

对一个验证请求 q ，定义：

$V(B)$

$C(q)$,

其中：

- y 是验证结果；
- $B^{q,y}$ 是验证后的状态；
- $C(q)$ 是工具、时间或 token 成本。

理想策略为：

$\arg\max_q$

$\operatorname{VOI}(q)$,

并且仅当：

$\max_q \operatorname{VOI}(q) > 0$

时执行验证。第一版实现不一定需要显式训练精确 VOI 模型，可以：

- 固定每条轨迹的验证预算；
- 使用验证工具选择监督；
- 通过偏好优化或约束 RL 减少无效验证。

理论作用是解释： **HOLD** 不等于总是验证，验证应由预期收益和成本共同决定。

7. 与现有工作的创新边界

7.1 与 ABBEL、Agent-BRACE 的区别

ABELL 和 Agent-BRACE 主要关注：

如何表示、压缩或维护一个供 Agent 行动使用的 belief state?

SIEVE 关注：

一条新观察是否有资格改变已经形成的可信状态？

区别如下：

维度	ABELL / Agent-BRACE	SIEVE
核心问题	belief representation / bottleneck	admissible observation-to-state transition
输入重点	历史轨迹与环境状态	当前可信状态、新观察及其元信息
更新方式	observation 后生成 posterior belief	先判断 UPDATE/HOLD/IGNORE，再执行受约束 patch
冲突处理	多依赖生成式 belief update	显式 HOLD，不覆盖旧值
局部性	主要由模型生成行为决定	Executor 提供字段级硬不变量
验证	非核心机制	HOLD 分支的选择性后续行为
贡献位置	belief 如何表示	evidence 是否可进入 belief

SIEVE 可以直接使用 ABDEL 或 Agent-BRACE 的状态表示，其贡献位于 belief updater 内部或之前，而不是再提出一种相似的 belief representation。

7.2 与记忆管理方法的区别

Memory-as-Action、Agentic Memory 等方法主要学习：

- 写入什么；
- 删除什么；
- 压缩什么；
- 检索什么；
- 将哪些内容保留在工作记忆中。

这些操作控制的是信息存取。

SIEVE 控制的是：

已经出现的新证据是否可以成为授权后续行动的可信状态。

因此，UPDATE 不是普通的 memory write；HOLD 允许信息保留在 evidence ledger 中，却不允许其进入 trusted state。

7.3 与 ReDAct 等延迟方法的区别

ReDAct 等 learning-to-defer 方法主要决定：

- 当前动作是否交给更强模型或专家；
- 当前预测是否应当延迟或升级。

SIEVE 的 `HOLD` 发生在更早的位置：

Observation → `State Admission` → Action.

它首先判断具体哪条证据影响哪个状态字段，再决定是否验证或暂缓依赖该字段的动作。

7.4 与 Prompt Filter 或 Verifier 的区别

简单过滤器通常输出 keep/drop，无法表示：

- 新证据可能有效，但需要验证；
- 旧可信值应该保留，候选值应隔离；
- 只允许局部字段变化；
- 同一不确定状态对低风险与高风险动作具有不同影响。

Action Verifier 则通常在动作生成后检查动作是否合法。SIEVE 的作用点更早，它试图在错误状态形成之前阻断污染。

8. 数据构造

8.1 单步 SFT 数据

每条样本包含：

代码块

```
1  {
2    "goal": "确认订单是否可以发货",
3    "belief_state": [
4      {
5        "id": "payment_status",
6        "value": "unpaid",
7        "status": "trusted",
8        "source": "official_payment_api",
9        "time": "2026-07-15T14:20:00+08:00"
10     }
11   ],
12   "observation": {
13     "content": "该订单已经支付",
14     "source": "cached_web_page",
```

```
15     "observed_time": "2026-07-15T14:31:00+08:00",
16     "valid_time": "unknown"
17 },
18 "candidate_action": {
19     "name": "ship_order",
20     "risk": "high"
21 },
22 "target": {
23     "decision": "HOLD",
24     "affected_fields": ["payment_status"],
25     "patch": [
26         {
27             "op": "SET_STATUS",
28             "id": "payment_status",
29             "value": "pending_verification"
30         }
31     ],
32     "verification_request": {
33         "tool": "official_payment_api",
34         "field": "payment_status"
35     }
36 }
37 }
```

8.2 配对反事实数据

同一个基础轨迹生成成对样本：

- 官方 API 的最新支付状态 → UPDATE ；
- 缓存网页的未知时间状态 → HOLD ；
- 上个月的支付记录 → IGNORE ；
- 另一个订单的支付状态 → IGNORE ；
- 与可信状态冲突且存在官方查询工具 → HOLD + Verify ；
- 与可信状态冲突但动作完全可逆、风险极低 → 可根据任务策略学习更宽松的决策。

应按基础 scenario 划分 train/dev/test，避免同一模板出现在不同数据集造成泄漏。

8.3 轨迹级数据

轨迹数据应包含：

- 多步 world state；

- 每一步 observation;
- revision decision;
- state patch;
- action;
- 工具成本;
- 最终任务成功;
- 不安全动作;
- 状态污染传播长度。

尤其需要包含延迟后果案例：

早期错误更新不会立即导致失败，但会在若干步后改变规划和动作。

这是第二阶段训练真正有意义的场景。

9. 实验设计

9.1 三层评测

Step Level

- decision macro-F1;
- UPDATE / HOLD / IGNORE precision 与 recall;
- False Update Rate;
- Missed Update Rate;
- query selection accuracy;
- calibration error。

State Level

- State-World Consistency;
- affected-set F1;
- collateral edit rate;
- unaffected state preservation;
- stale-state retention rate;
- recovery latency;
- contamination propagation length。

Trajectory Level

- Task Success Rate;
 - unsafe / irreversible action rate;
 - verification tool calls;
 - total tool calls;
 - token 与 latency cost;
 - fixed-budget success;
 - clean-to-noisy degradation;
 - safety–utility Pareto frontier。
-

9.2 必须设置的 Baseline

1. Frozen Vanilla Agent;
2. Strong Prompt-only SIEVE;
3. Rule-based Admission Gate;
4. Retrieval Filter / Reranker;
5. Free-form Belief Rewrite;
6. ABBEL-style Posterior Update;
7. Agent-BRACE-style Atomic Belief;
8. Always Update;
9. Always Hold / Always Verify;
10. Oracle Revision Policy。

所有系统必须统一：

- backbone;
 - tool set;
 - context budget;
 - token budget;
 - verification budget;
 - action model。
-

9.3 核心消融

- 去掉 provenance;
 - 去掉 time;
 - 去掉 action risk;
 - 去掉 affected-field prediction;
 - free-form rewrite 替换 typed patch;
 - 允许 action model 读取原始 observation;
 - SFT vs SFT + Preference;
 - SFT vs SFT + Constrained RL;
 - fixed verification threshold vs learned query selection;
 - 去掉确定性 Executor。
-

10. 权重与约束敏感性实验

10.1 SFT 权重敏感性

对以下参数分别进行扫描：

α_A , α_P , α_Q .

建议对每项使用：

0.25, 0.5, 1, 2, 4.

重点观察：

- decision accuracy;
- affected-set recall;
- patch validity;
- query selection;
- trajectory success。

若某一辅助损失权重稍微变化便造成结果剧烈波动，说明联合任务设计不稳。

10.2 固定奖励权重实验

虽然主方法推荐使用约束优化，但仍应提供固定加权奖励作为对照：

`\lambda_{\mathrm{fu}} C_{\mathrm{false\ update}}`

`\lambda_v C_{\mathrm{verify}}`

`\lambda_u C_{\mathrm{unsafe}}`

`\lambda_h C_{\mathrm{hold}}`.

 无效公式

- task success–false update 曲线；
- task success–verification cost 曲线；
- safety–utility Pareto frontier；
- hold rate–completion rate 曲线。

目的不是寻找一组“最好看”的权重，而是展示固定加权和确实存在较强敏感性，从而支持约束优化设计。

10.3 约束预算敏感性

建议扫描：

参数	建议取值
$\$ \backslash epsilon_{\mathrm{fu}} \$$	0.01、0.03、0.05、0.10
$\$ \backslash epsilon_{\mathrm{safe}} \$$	0、0.01、0.03、0.05
$\$ B_{\mathrm{verify}} \$$	0、1、2、4 次/轨迹
$\$ B_{\mathrm{hold}} \$$	0.05、0.10、0.20、0.40

重点回答：

- 更严格的污染约束是否一定降低任务完成率；
- 验证预算增加是否存在边际收益递减；
- **HOLD** 预算过小是否导致过度更新；
- **HOLD** 预算过大是否导致不行动；
- 训练后的拉格朗日乘子是否稳定收敛。

10.4 统计要求

- 至少 3 个随机种子；
 - 报告 bootstrap 95% 置信区间；
 - 同一 case 的模型比较使用 paired bootstrap 或 McNemar test；
 - 超参数只在 dev set 选择，test set 固定；
 - 不只报告单个最优点，应报告 Pareto frontier；
 - 报告失败类型分布，防止平均分掩盖退化策略。
-

11. 最小可行实现

11.1 模型结构

推荐：

代码块

```
1  Frozen Base LLM
2      +
3  Revision LoRA / Adapter
4      +
5  Deterministic State Executor
6      +
7  Frozen Action Agent
```

Revision Adapter 可以与 Action Agent 共享基础模型参数，但使用不同 Adapter。这样可以降低训练成本，并保持功能分离。

11.2 执行流程伪代码

代码块

```
1  def sieve_step(
2      belief_state,
3      observation,
4      metadata,
5      goal,
6      candidate_action
7  ):
8      decision = revision_policy.predict(
9          belief_state=belief_state,
10         observation=observation,
```

```
11         metadata=metadata,
12         goal=goal,
13         action_risk=candidate_action.risk
14     )
15
16     updated_state = state_executor.apply(
17         belief_state=belief_state,
18         decision=decision
19     )
20
21     if (
22         candidate_action.risk == "high"
23         and state_executor.has_pending_dependency(
24             updated_state,
25             candidate_action
26         )
27     ):
28         return updated_state, decision.verification_request
29
30     action = action_agent.act(
31         belief_state=updated_state,
32         goal=goal
33     )
34
35     return updated_state, action
```

11.3 最小实验路线

1. 构造 300–500 条人工核验 paired cases;
2. 比较 Vanilla、Prompt-only、Rule Gate 和 Oracle;
3. 确认 Prompt 与 Oracle 之间确实存在稳定差距;
4. 用 10k–30k 条反事实 step data 训练 Revision LoRA;
5. 验证 Step 指标是否改善;
6. 把模型放入多步 Agent 环境，检查是否出现过度 **HOLD** 或延迟信用问题;
7. 只有确认存在轨迹取舍问题后，再进行 Preference 或 Constrained RL;
8. 最终扩展到多个 benchmark 和 OOD stress test。

12. 整体框架图

代码块Lowchart LR

```
2      O[环境 / 用户 / 工具<br/>新观察 o_t] --> M[观察元信息<br/>来源、时间、条件]
3      B[可信任务状态 B_t-1] --> R
4      M --> R
5      O --> R
6      G[任务目标 g_t<br/>候选动作风险 r_t] --> R
7
8      subgraph S[SIEVE: 可学习的 State Revision Policy]
9          R[Revision Adapter] --> D{状态修正决策}
10         D -->|UPDATE| U[受影响字段<br/>局部 JSON Patch]
11         D -->|HOLD| H[保留旧可信值<br/>标记待确认]
12         D -->|IGNORE| I[不修改可信状态]
13         H --> Q{验证是否有价值}
14         Q -->|是| V[生成定向验证请求]
15         Q -->|否| W[暂缓依赖该字段的<br/>高风险动作]
16     end
17
18     U --> E[确定性 State Executor]
19     H --> E
20     I --> E
21     E --> NB[更新后的可信任务状态 B_t]
22     NB --> A[冻结的 Action Agent]
23     A --> X[环境动作 a_t]
24     V --> O
25     X --> O
26
27     T1[阶段一: SFT<br/>学习单步裁决、字段定位和 Patch] -.训练.-> R
28     T2[阶段二: 轨迹优化<br/>学习任务成功、污染风险与验证成本的取舍] -.优化同一策略.-> R
```

不支持 Mermaid 的编辑器可使用以下线性版本：

代码块

```
1  环境 / 用户 / 工具观察
2      ↓
3  当前可信任务状态 + 来源/时间/条件 + 任务目标 + 动作风险
4      ↓
5      Revision Policy
6      ↓
7  ┌───────────┴───────────┐
8  UPDATE      HOLD      IGNORE
9      ↓         ↓         ↓
10  局部Patch  保留旧值  不修改状态
11      ↓
12  验证或暂缓高风险动作
13      ↓
```

```
14      确定性 State Executor
15      ↓
16      更新后的可信任任务状态
17      ↓
18      冻结的 Action Agent
19      ↓
20      环境动作
```

13. 论文贡献与主张

13.1 一句话主张

Retrieval determines what an agent sees; SIEVE determines which observed evidence is allowed to revise the trusted task state that governs subsequent actions.

中文：

检索决定 Agent 看见什么；SIEVE 决定哪些已观察证据有资格修改后续行动所依赖的可信任任务状态。

13.2 三个贡献点

- 问题定义：** 提出观察诱导的状态污染问题，并使用 false update、状态一致性、传播长度和下游不安全动作进行直接评测。
- 方法设计：** 提出一个独立的 State Revision Policy，在 UPDATE / HOLD / IGNORE 之间进行观察准入决策，并通过 typed local patch 和确定性 Executor 保证局部性与未采纳信息隔离。
- 训练与验证：** 先通过 SFT 学习稳定的单步状态修正，再在确有长期取舍问题时使用偏好优化或约束 RL，在相同 backbone 和验证预算下比较 Prompt、belief bottleneck 与 always-verify 等强基线。

14. 审稿人可能提出的问题

Q1：这是不是 Prompt 工程？

回答重点：

- Prompt-only 是强基线；
- 方法训练的是稳定的 observation-to-state transition policy；
- 需要证明在语言改写、长轨迹和固定预算下，训练策略比 Prompt 更少 false update 和无效验证；

- typed executor 提供 Prompt 无法稳定保证的结构不变量。

Q2: 这是不是简单的 memory filter?

回答重点:

- 信息可以保留在 evidence ledger, 但不进入 trusted state;
- `HOLD` 不是 drop, 而是候选值隔离和可选验证;
- 方法输出局部状态 patch, 而不只是 keep/drop;
- 最终评测包含状态传播和下游行动。

Q3: ABBEL 已经做了 belief RL, 创新在哪里?

回答重点:

- ABBEL 研究 belief bottleneck 和 posterior belief generation;
- SIEVE 研究观察是否具有修改 trusted state 的准入资格;
- SIEVE 允许 evidence 被观察但不被 committed;
- Executor 对局部更新和未采纳证据提供硬约束;
- action backbone 固定, 收益归因于 state transition。

Q4: 为什么一定需要 RL?

回答重点:

- 不预设一定需要;
- 先进行 Prompt 和 SFT 诊断;
- 只有当过度验证、延迟信用和长期风险权衡真实存在时才使用轨迹优化;
- 若 Prompt 或 SFT 已解决问题, RL 不应作为核心贡献。

Q5: 理论是不是建立在分类器永远正确的假设上?

回答重点:

- 理论保证来自 Executor, 不来自模型永远正确;
- 模型错误通过 false-update rate、affected-set error 显式进入风险上界;
- 论文明确区分结构保证和学习误差。

15. 研究边界

本文不声称解决:

- 所有长期记忆管理问题；
- 完整世界模型学习；
- 通用事实核查；
- 全部 prompt injection 攻击；
- 全局最优规划；
- 开放世界中的绝对真实性判断。

本文只研究：

在给定当前任务状态、新观察及其元信息时，如何学习一个稳定、局部、成本敏感的观察准入与状态修正策略。

16. 核心参考文献入口

- [STALE: Can LLM Agents Know When Their Memories Are No Longer Valid?](#)
- [MemConflict: Evaluating Long-Term Memory Systems Under Memory Conflicts](#)
- [ABBEL: LLM Agents Acting through Belief Bottlenecks Expressed in Language](#)
- [Agent-BRACE: Decoupling Beliefs from Actions in Long-Horizon Tasks via Verbalized State Uncertainty](#)
- [ReDAct: Uncertainty-Aware Deferral for LLM Agents](#)
- [UProp: Investigating the Uncertainty Propagation of LLMs in Multi-Step Agentic Decision-Making](#)
- [Sequential Learning-to-Defer](#)
- [\$\tau\$ -bench: Tool-Agent-User Interaction in Real-World Domains](#)
- [AgentDojo](#)
- [MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents](#)
- [ALFWorld](#)

注：2026 年文献目前应按预印本谨慎表述。正式投稿前需要重新核验版本、作者列表、录用状态与引用信息。